**Name:** Nilanjan Ghosh
**Register Number:** RA2311033010002
**Course:** Cognizant Digital Nurture 4.0
**Week:** 2
**Topic:** Spring Core – Basic Spring Application

### Objective

To create a basic Spring application using XML configuration and Dependency Injection.

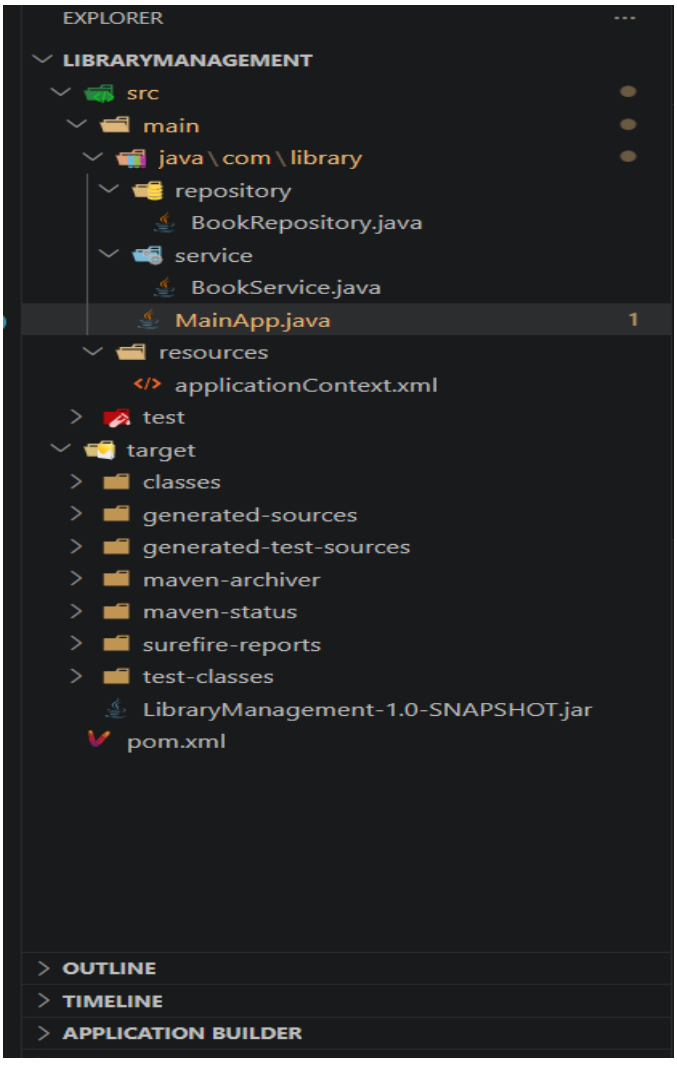
---

### Software Requirements

- Java JDK 17
- Apache Maven
- Spring Framework
- Visual Studio Code

---

### Project Structure




.

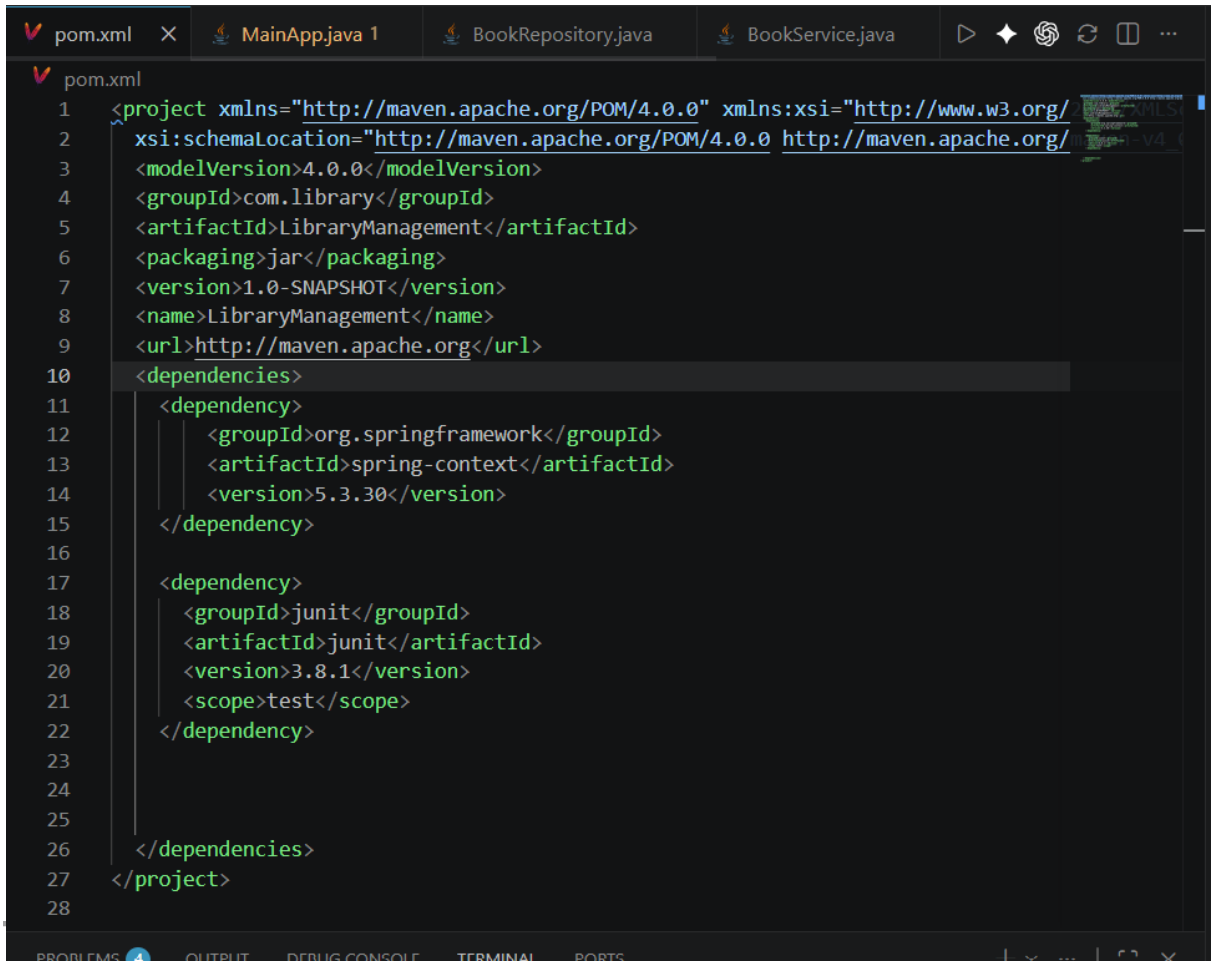
**Step 1: Configure Maven Project**

**pom.xml**

code.

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.3.30</version>
</dependency>
```

**Screenshot 2:** pom.xml




**Step 2: Create Repository Class**

**BookRepository.java**

code:

```
package com.library.repository;

public class BookRepository {
```

```
    public void displayBook() {
        System.out.println("Book Repository Called");
    }
}
```

---

**Step 3: Create Service Class**

**BookService.java**

code:

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void showBookDetails() {
        System.out.println("Book Service Called");
        bookRepository.displayBook();
    }
}
```

---

**Step 4: Configure Spring XML**

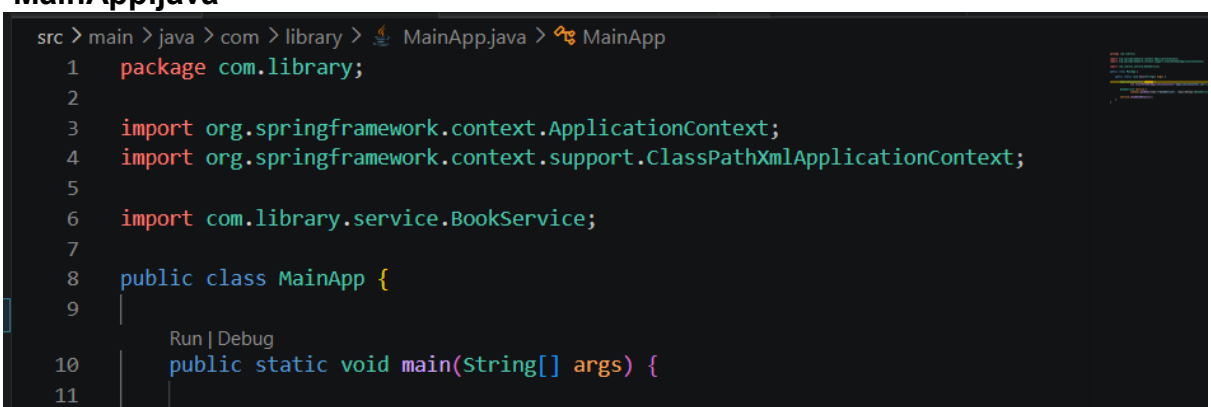
**applicationContext.xml**

code.

---

**Step 5: Main Class**

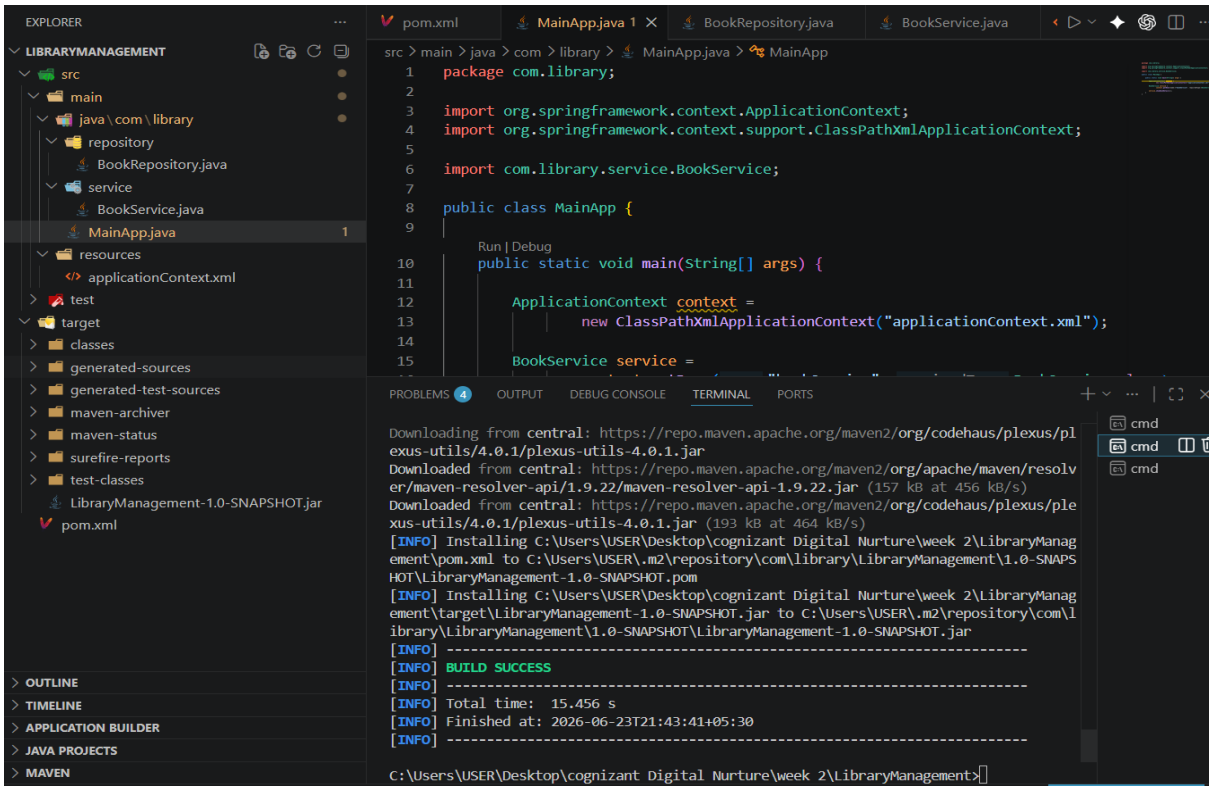
**MainApp.java**



## Step 6: Build and Run

Open Terminal:

mvn clean install

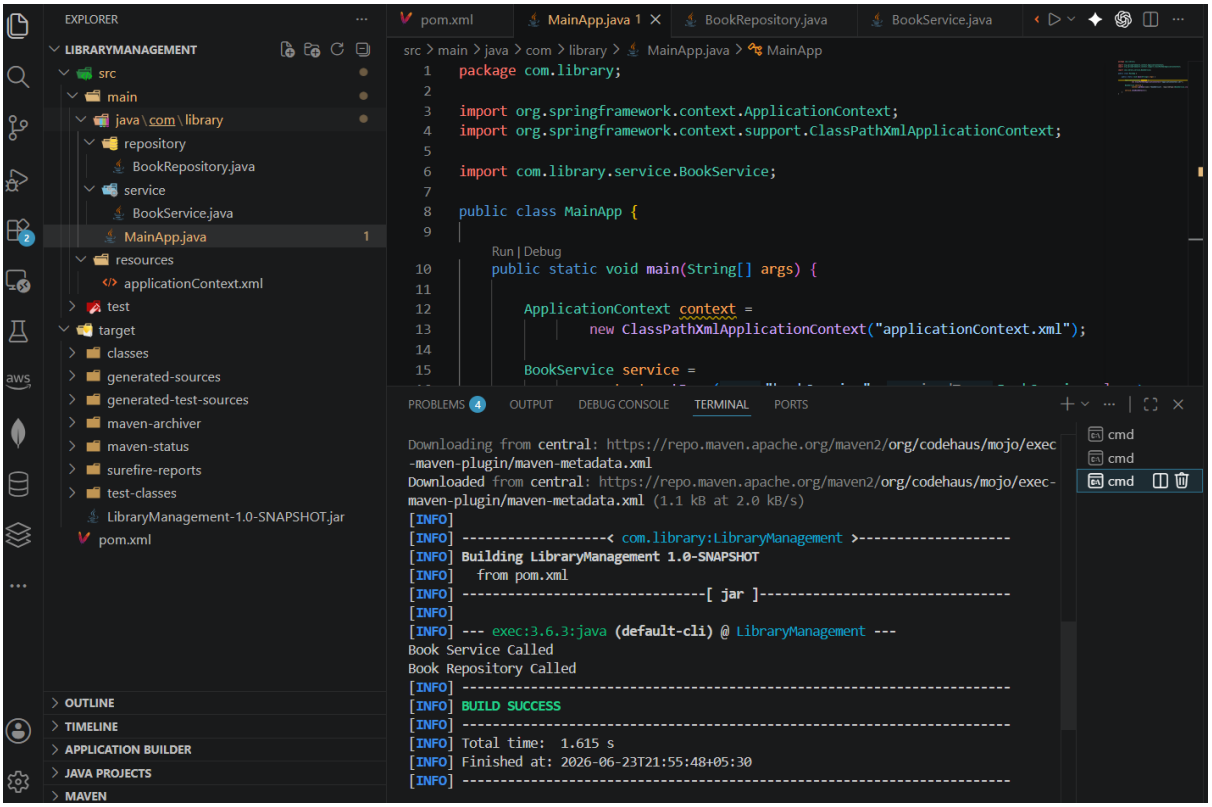



## Output

Book Service Called
Book Repository Called

Insert output screenshot.

**Screenshot 9:** Final Output




---

## Result

The Spring application was successfully configured using XML-based configuration. The Spring Container created and managed the beans, and Dependency Injection was implemented successfully.

## Exercise 2: Implementing Dependency Injection

### Aim

To implement Dependency Injection (DI) using Spring Framework's Inversion of Control (IoC) container.

---

### Objective

The objective of this exercise is to inject the BookRepository dependency into the BookService class using setter-based Dependency Injection through the Spring XML configuration.

---

### Software Requirements

- Java Development Kit (JDK)
- Apache Maven
- Spring Core Framework
- Visual Studio Code

---

### Theory

Dependency Injection (DI) is a design pattern in which the Spring IoC container provides the required objects to a class instead of the class creating them manually.

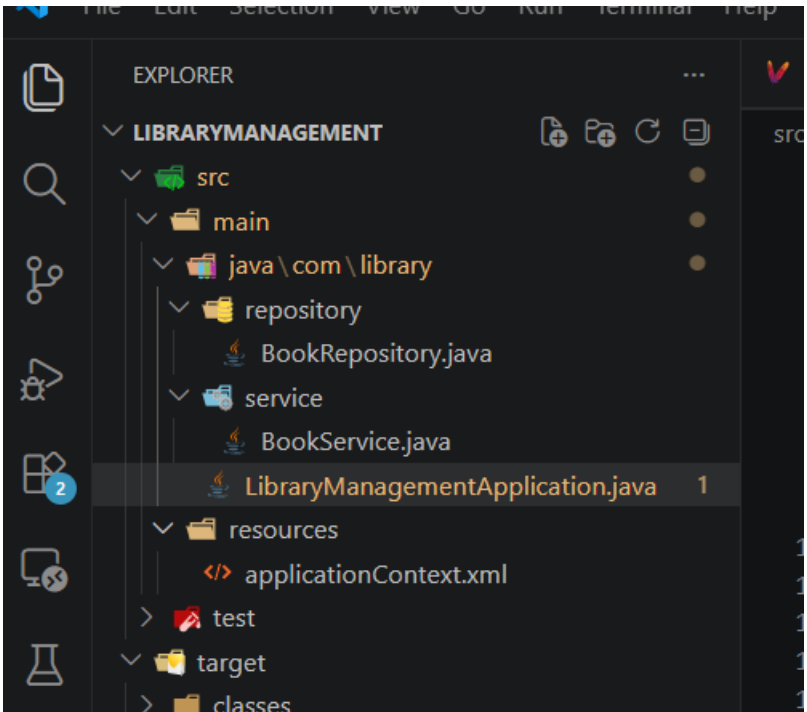
In this exercise, the BookService class depends on the BookRepository class. Spring injects the BookRepository bean into BookService using setter injection configured in the applicationContext.xml file.

---

### Procedure

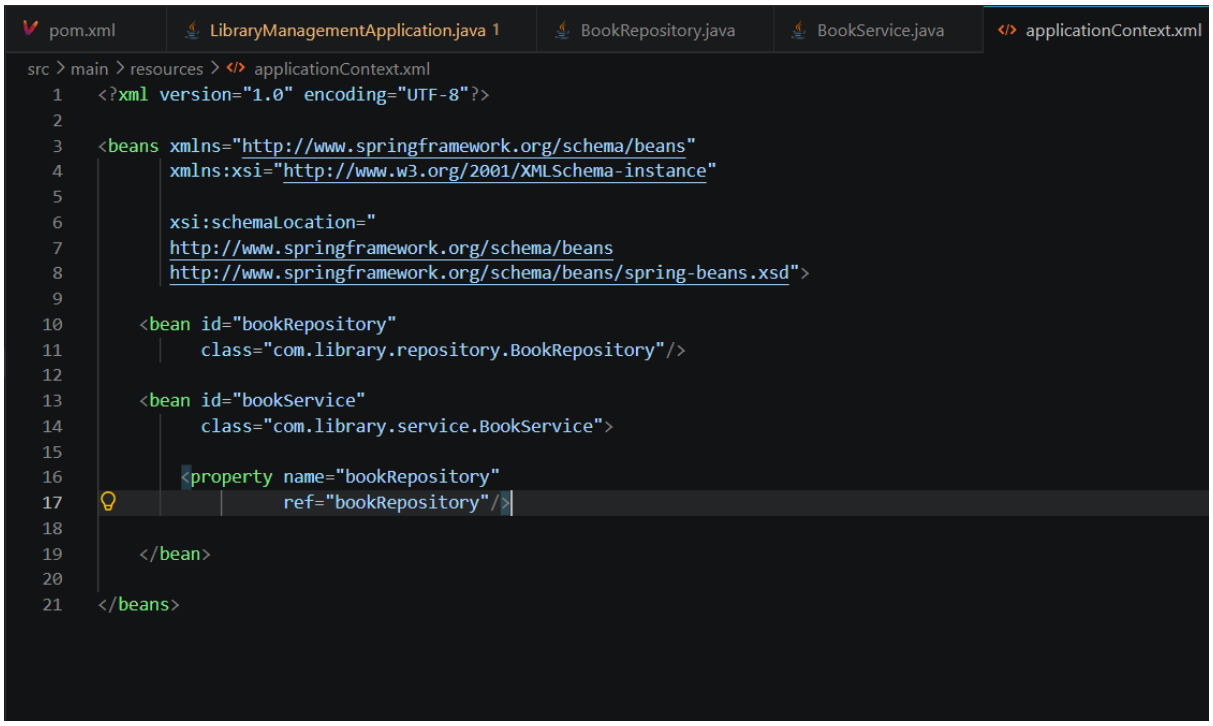
#### Step 1

Modified the BookService class by adding a setter method for BookRepository.



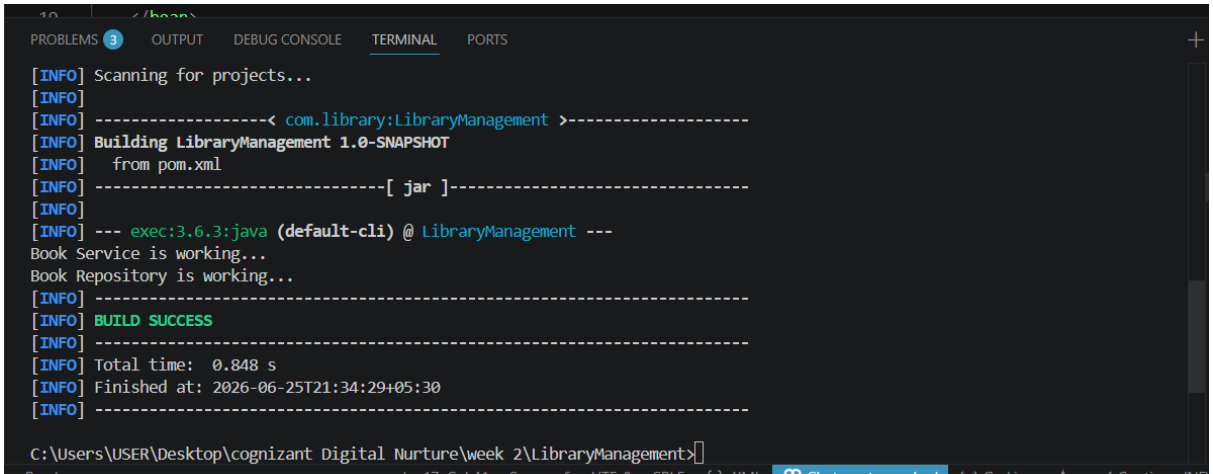
## Step 2

Updated the applicationContext.xml file by defining the BookRepository bean and injecting it into the BookService bean using the <property> tag.




## Step 3

Executed the LibraryManagementApplication class to verify Dependency Injection.



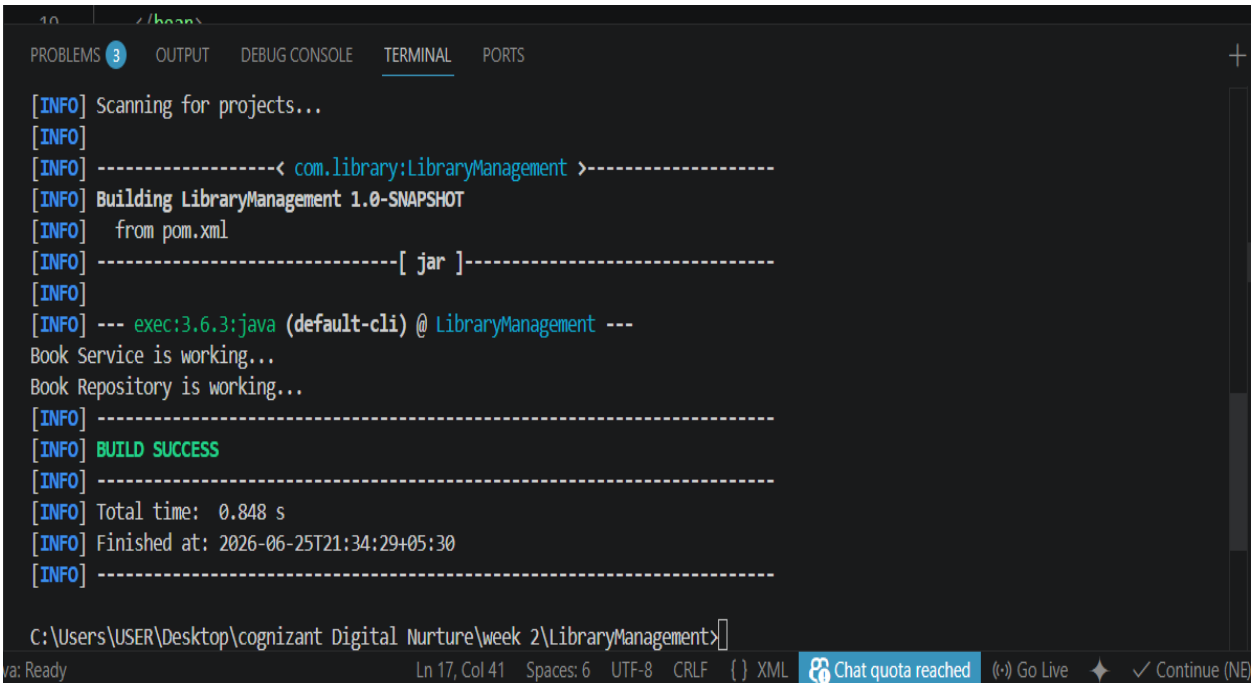
---

### Output

The application executed successfully and displayed:

Book Service is working...

Book Repository is working...




---

### Result

The BookRepository dependency was successfully injected into the BookService class using Spring's setter-based Dependency Injection.

---

### Conclusion

This exercise demonstrated how Spring's IoC container manages object creation and dependency injection through XML configuration. Setter-based Dependency Injection

improves modularity, maintainability, and loose coupling between application components.

### Exercise 4: Creating and Configuring a Maven Project

#### Aim

To create and configure a Maven project for a Spring-based Library Management application.

---

#### Objective

The objective of this exercise is to configure a Maven project by adding the required Spring Framework dependencies and configuring the Maven Compiler Plugin for Java 1.8.

---

**Software Requirements**

- Java Development Kit (JDK)
- Apache Maven
- Visual Studio Code
- Spring Framework

---

**Theory**

Apache Maven is a build automation and dependency management tool used for Java projects. It simplifies project management by automatically downloading and managing required libraries through the pom.xml file.

In this exercise, Spring Framework dependencies were added to support application context management, aspect-oriented programming, and web application development.

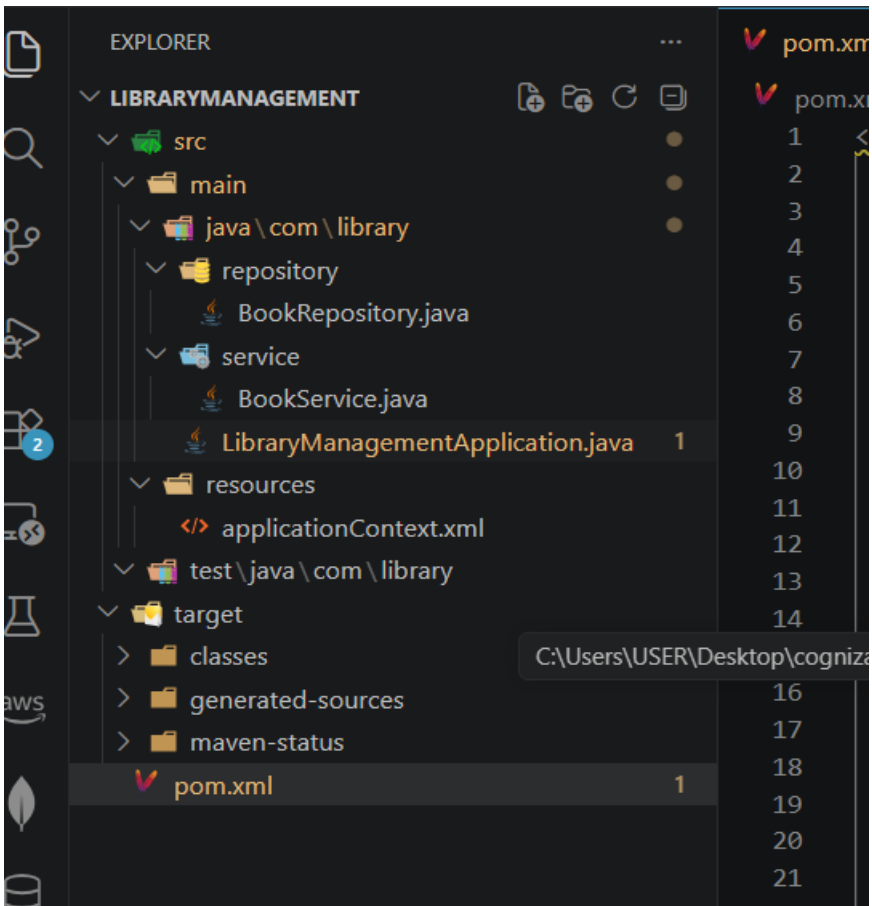
The Maven Compiler Plugin was configured to compile the project using Java 1.8.

---

**Procedure**

**Step 1**

Opened the existing Maven project named **LibraryManagement**.



**Step 2**

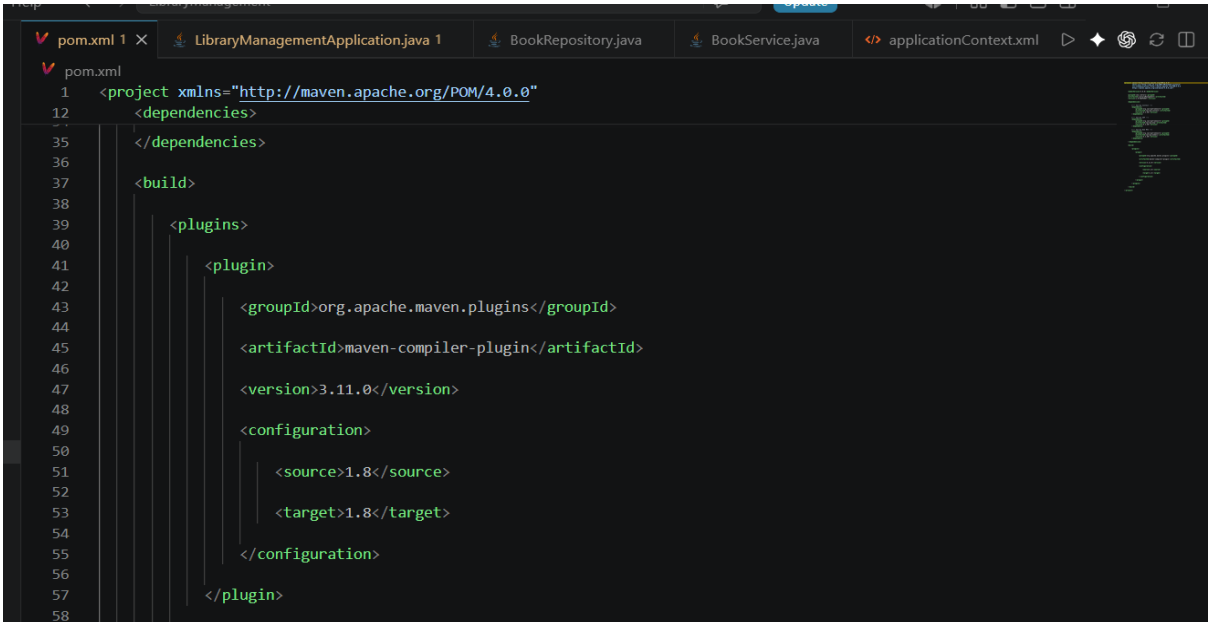
Updated the pom.xml file by adding the following Spring Framework dependencies:

- Spring Context
- Spring AOP
- Spring WebMVC

```
pom.xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
    <version>1.0-SNAPSHOT</version>

    <dependencies>

        <!-- Spring Context -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>5.3.30</version>
        </dependency>

        <!-- Spring AOP -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-aop</artifactId>
            <version>5.3.30</version>
        </dependency>

        <!-- Spring Web MVC -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-webmvc</artifactId>
            <version>5.3.30</version>
        </dependency>

    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
```

**St**

Co target versions.



```
pom.xml 1 | LibraryManagementApplication.java 1 | BookRepository.java | BookService.java | applicationContext.xml
pom.xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
    <dependencies>
    </dependencies>

    <build>

        <plugins>

            <plugin>

                <groupId>org.apache.maven.plugins</groupId>

                <artifactId>maven-compiler-plugin</artifactId>

                <version>3.11.0</version>

                <configuration>

                    <source>1.8</source>

                    <target>1.8</target>

                </configuration>

            </plugin>
```

### Step 4

Executed the Maven build command to download dependencies and verify the project configuration.

```
[INFO] --- compiler:3.11.0:compile (default-compile) @ LibraryManagement ---
[INFO] Changes detected - recompiling the module! :source
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 3 source files with javac [debug target 1.8] to target\classes
[WARNING] bootstrap class path not set in conjunction with -source 8
[WARNING] source value 8 is obsolete and will be removed in a future release
[WARNING] target value 8 is obsolete and will be removed in a future release
[WARNING] To suppress warnings about obsolete options, use -Xlint:-options.
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.163 s
[INFO] Finished at: 2026-06-25T22:01:36+05:30
[INFO] ------------------------------------------------------------------------
```

### Output

The Maven project was successfully built.

BUILD SUCCESS


```
[INFO] --- compiler:3.11.0:compile (default-compile) @ LibraryManagement ---
[INFO] Changes detected - recompiling the module! :source
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 3 source files with javac [debug target 1.8] to target\classes
[WARNING] bootstrap class path not set in conjunction with -source 8
[WARNING] source value 8 is obsolete and will be removed in a future release
[WARNING] target value 8 is obsolete and will be removed in a future release
[WARNING] To suppress warnings about obsolete options, use -Xlint:-options.
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.163 s
[INFO] Finished at: 2026-06-25T22:01:36+05:30
[INFO] ------------------------------------------------------------------------
```

---

**Result**

The Maven project was successfully configured with the required Spring dependencies and the Maven Compiler Plugin for Java 1.8.

---

**Conclusion**

This exercise demonstrated how Maven simplifies dependency management and project configuration. By adding the required Spring modules and configuring the compiler plugin, the project was successfully prepared for Spring application development.